

Mars Exploration:-

Code:-

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static int marsExploration(String s) {
        int count = 0;
        String message = "SOS";
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != message.charAt(i % 3)) {
                count++;
            }
        }
        return count;
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

        int result = Result.marsExploration(s);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();
    }
}
```

```
bufferedReader.close();  
bufferedWriter.close();  
}  
}
```

Output:-

✓ **Test case 0**

✓ Test case 1

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

1 **S0SSPSSQSSOR**

[Download](#)

Expected Output

1 **3**

[Download](#)

Find Anagrams:-

Code:-

```
import java.io.*;
import java.util.*;
class Solution {
    public static List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        if (s.length() < p.length()) {
            return result;
        }
        int[] pCount = new int[26];
        int[] windowCount = new int[26];
        for (char c : p.toCharArray()) {
            pCount[c - 'a']++;
        }
        int k = p.length();
        for (int i = 0; i < s.length(); i++) {
            windowCount[s.charAt(i) - 'a']++;
            if (i >= k) {
                windowCount[s.charAt(i - k) - 'a']--;
            }
            if (Arrays.equals(pCount, windowCount)) {
                result.add(i - k + 1);
            }
        }
        return result;
    }
    public static void main(String[] args) throws IOException {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));
        String s = br.readLine();
        String p = br.readLine();
        List<Integer> result = findAnagrams(s, p);
```

```

        System.out.println(result);
    }
}

```

Output:-

The screenshot displays the LeetCode submission interface for a problem. The submission is marked as "Accepted" with 65/65 testcases passed. The user "tameem_shaik_" submitted it on Oct 05, 2026 at 12:07. A promotional banner for "Unlock the Full LeetCode Experience" is visible, offering a discount of ₹583.25. The performance metrics show a runtime of 10 ms (Beats 80.43%) and memory usage of 47.00 MB (Beats 46.71%). A bar chart illustrates the runtime distribution, with the user's solution being the fastest. The code is written in Java and implements a function to find anagrams. The test result shows two cases passed: Case 1 with input s="cbaebabacd" and p="abc", and Case 2 with input s="cbaebabacd" and p="abc".

Accepted 65 / 65 testcases passed
 tameem_shaik_ submitted at Oct 05, 2026 12:07

Runtime
 10 ms | Beats 80.43%

Memory
 47.00 MB | Beats 46.71%

Code
 Java

```

21         result.add(1 - k + 1);
22     }
23 }
24 return result;
25 }
26 public static void main(String[] args) throws IOException {
27     BufferedReader br =
28         new BufferedReader(new InputStreamReader(System.in));
29     String s = br.readLine();
30     String p = br.readLine();
31     List<Integer> result = findAnagrams(s, p);
32     System.out.println(result);
33 }
34 }

```

Testcase | **Test Result**

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

s =
 "cbaebabacd"

p =
 "abc"

Output